

Introduction to NP- Hard and NP- Complete Problems

<https://www.youtube.com/watch?v=e2cF8a5aAhE>

Context of Research in Algorithms

Algorithms can be classified based on their time complexity:

1. Polynomial-Time Algorithms:

- These are efficient algorithms with runtimes like $O(n)$, $O(\log n)$, $O(n \log n)$, $O(n^2)$, etc.
- Examples:
 - **Searching Algorithms:** Linear Search ($O(n)$), Binary Search ($O(\log n)$).
 - **Sorting Algorithms:** Bubble Sort ($O(n^2)$), Merge Sort ($O(n \log n)$).
- Researchers aim to develop faster algorithms, e.g., searching algorithms with $O(1)$ complexity or sorting algorithms with $O(n)$ complexity.

Context of Research in Algorithms

Algorithms can be classified based on their time complexity:

2. Exponential-Time Algorithms:

- These algorithms have runtimes like $O(2^n)$, $O(n^n)$, $O(n \cdot 2^n)$, etc.
- Examples:
 - Subset Sum Problem ($O(2^n)$).
 - Traveling Salesperson Problem (TSP) ($O(n \cdot 2^n)$).
- The goal is to find more efficient, polynomial-time solutions to these problems because exponential-time algorithms become infeasible for large n .

NP-Hard and NP-Complete

Polynomial Time

Linear Search — n

Binary Search — $\log n$

Insertion Sort — n^2

Merge Sort — $n \log n$

Matrix Multiplication — n^3

Exponential Time

0/1 Knapsack — 2^n

Traveling SP — 2^n

Sum of Subsets — 2^n

Graph Coloring — 2^n

Hamiltonian Cycle — 2^n

Why This is a Research Area

- Exponential-time algorithms grow far faster than polynomial-time algorithms.
 - Even $O(n^{10})$ or $O(n^{100})$ is smaller than $O(2^n)$ for sufficiently large n .
- For many problems, no polynomial-time solution has been found, making this an active area of research for computer scientists and mathematicians.

NP-Hard and NP-Complete: A Framework for Research

Since finding solutions for these problems has been challenging, a framework called **NP-Hard** and **NP-Complete** was introduced. This framework provides:

- **Guidelines:** To categorize problems and focus research efforts.
- **Goals:** To determine if exponential-time problems can be solved in polynomial time.

Despite significant research, no polynomial-time solutions have been discovered for many problems, making this an ongoing field of study.

Key Ideas Behind NP-Hard and NP-Complete Problems

To understand NP-Hard and NP-Complete problems, there are two main ideas that guide research in this area:

1. Establishing Relationships Between Problems:

- When finding a polynomial-time solution for a problem is difficult, the goal is to identify relationships between different problems.
- By showing similarities and associations, it becomes possible to demonstrate that solving one problem can help solve others.
 - Example: Instead of separately working on the **Knapsack Problem** and **Graph Coloring Problem**, researchers aim to show they share similar properties.
 - If one problem is solved, it can lead to solutions for related problems.
- This approach avoids redundant effort and focuses on solving problems collectively.

Key Ideas Behind NP-Hard and NP-Complete Problems

To understand NP-Hard and NP-Complete problems, there are two main ideas that guide research in this area:

2. Exploring Non-Deterministic Algorithms:

- If a polynomial-time **deterministic algorithm** (an algorithm that follows a fixed sequence of steps) cannot be created, consider developing a polynomial-time **non-deterministic algorithm** (an algorithm that explores multiple possibilities simultaneously).
- Non-deterministic algorithms provide a different perspective for approaching complex problems, even if deterministic solutions remain elusive. (**elusive** - difficult to find)

Let us `first` see

`2.` Non-Deterministic Algorithms: An Overview

2. Non-Deterministic Algorithms: An Overview

What are Non-Deterministic Algorithms?

- Most algorithms we use are **deterministic**, meaning:
 - We know exactly how every statement in the algorithm works.
 - The behavior is predictable and follows a clear sequence of steps.
- In contrast, a **non-deterministic algorithm** includes parts where:
 - We do not specify how certain steps work.
 - These steps are left "open" to be determined later, either by future research or advancements in understanding.

2. Non-Deterministic Algorithms: An Overview (cont)

Why Use Non-Deterministic Algorithms?

- When solving problems like the **0/1 Knapsack Problem**, some parts of the algorithm may be straightforward, while others may be unclear or difficult to express in polynomial time.
- Instead of discarding incomplete work, these unclear parts are marked as **non-deterministic**, allowing:
 - **Preservation of research progress:** Future researchers can revisit and refine these parts.
 - **Collaboration:** Other researchers can complete the missing pieces, turning the algorithm into a deterministic one.

2. Non-Deterministic Algorithms: An Overview (cont..)

NP-Hard and NP-Complete

Nondeterministic

```
Algorithm NSearch(A, n, key)
{
    j = choice();
    if (key = A[j])
    {
        write(j);
        Success();
    }
    write(0);
    Failure();
}
```

Exponential Time

0/1 Knapsack — 2^n

Traveling SP — 2^n

Sum of Subsets — 2^n

Graph Coloring — 2^n

Hamiltonian Cycle — 2^n

2. Non-Deterministic Algorithms: An Overview (cont..)

```
Algorithm NSearch(A, n, key)
{ // Non-deterministically picks an index
  j = choice();
  if (key == A[j]) {
    write(j);
    success();
  }
  write(0);
  failure();
}
```

Deterministic vs. Non-deterministic View:

In Deterministic Execution:

You'd check each element from `A[0]` to `A[4]` one by one — that's **linear search** ($O(n)$).

In Non-deterministic Execution:

The `choice()` magically “guesses” the correct index `j = 3` (where `A[3] = 9`) on the first try — **as if it knew in advance**.

This is like saying:

“If there exists any `j` such that `A[j] == key`, then just instantly jump to that value of `j`.”

It's not practical in real machines, but it models **how an NP machine would “guess” a solution path** and then verify it in **polynomial time**.

2. Non-Deterministic Algorithms: An Overview (cont..)

How Non-Deterministic Algorithms Work:

- Most parts of the algorithm remain deterministic.
- Non-deterministic parts are marked using specific constructs, such as:
 - **Choice:** Represents a decision point with multiple possibilities.
 - **Success/Failure:** Defines conditions for achieving or failing to find a solution.
- It is assumed that these non-deterministic steps take **one unit of time**, allowing for simplicity in theoretical analysis.

Non deterministic

```
Algorithm NSearch(A, n, key)
{
    j = choice();
    if (key = A[j])
    {
        write(j);
        Success();
    }
    write(0);
    Failure();
}
```

2. Non-Deterministic Algorithms: An Overview (cont...)

Example: Non-Deterministic Search Algorithm

- A non-deterministic search algorithm uses statements like:
 - **Choice:** Selects a path or value from multiple options.
 - **Success/Failure:** Determines whether a solution has been found.
- These non-deterministic statements serve as placeholders for parts of the algorithm that are not yet defined.

By **writing non-deterministic algorithms**, researchers can document their progress and provide a foundation for others to build upon, ensuring that efforts are **not wasted** even if the problem remains unsolved.

Non deterministic

```
Algorithm NSearch(A, n, key)
{
    j = choice();
    if (key = A[j])
    {
        write(j);
        Success();
    }
    write(0);
    Failure();
}
```

2. Non-Deterministic Search Algorithm Explained

What is the Idea?

- The non-deterministic search algorithm demonstrates how non-deterministic algorithms can theoretically perform faster than current deterministic ones.
- This example focuses on searching for a key in an array of size n in **constant time** ($O(1)$), compared to the fastest deterministic method, Binary Search, which takes $O(\log n)$.

Algorithm NSearch(A, n, key)

```
{  
  j = choice(); — 1  
  if (key = A[j])  
  {  
    write(j);  
    Success(); — 1  
  }  
  write(0);  
  Failure(); — 1  
}
```

$\frac{1}{O(1)}$

A	10	8	6	9	4	2
	1	2	3	4	5	6

key = 9

2. How the Non-Deterministic Search Algorithm Works

```
Algorithm NSearch(A, n, key)
{
    j = choice(); — 1
    if (key = A[j])
    {
        write(j);
        Success(); — 1
    }
    write(0);
    Failure(); — 1
}
O(1)
A [0|8|6|9|4|2]
  1 2 3 4 5 6   key = 9
```

1. The Process:

- The algorithm uses a special construct called **choice**, which directly "knows" the index j of the key in the array.
- It verifies whether the key is present at index j :
 - If yes, it declares the search successful.
 - If no, it declares the search unsuccessful, meaning the key does not exist in the array.

2. How the Non-Deterministic Search Algorithm Works: (con

2. What is Non-Deterministic?

- The statement **choice(j)** instantly identifies the correct index j without performing any search logic.
- How this is done is **unknown today**, which makes the algorithm **non-deterministic**.

3. Why Use This Approach?

- It serves as a placeholder for future research. If a method is discovered to implement **choice(j)** deterministically, the algorithm will become deterministic and achieve $O(1)$ time complexity.

```
Algorithm NSearch(A, n, key)
{
    j = choice(); — 1
    if (key = A[j])
    {
        write(j);
        Success(); — 1
    }
    write(0);
    Failure(); — 1
}

A [10 | 8 | 6 | 9 | 4 | 2]
   1  2  3  4  5  6
key = 9
```

$O(1)$

Classes P and NP: An Overview

To understand the framework for classifying problems, we define two important classes of algorithms: P and NP

Classes P and NP: An Overview

(cont...)

Class P : Deterministic Polynomial-Time Algorithms

- These are algorithms that are:
 - **Deterministic**, meaning we know exactly how each step works.
 - **Efficient**, as they solve problems in polynomial time ($O(n^k)$, where k is a constant).
- Examples:
 - Linear Search: $O(n)$
 - Binary Search: $O(\log n)$
 - Bubble Sort: $O(n^2)$
 - Merge Sort: $O(n \log n)$
 - Single Source Shortest Path (Dijkstra's Algorithm): $O(V^2)$ or $O(E + V \log V)$
 - Huffman Coding: $O(n \log n)$

Classes P and NP: An Overview

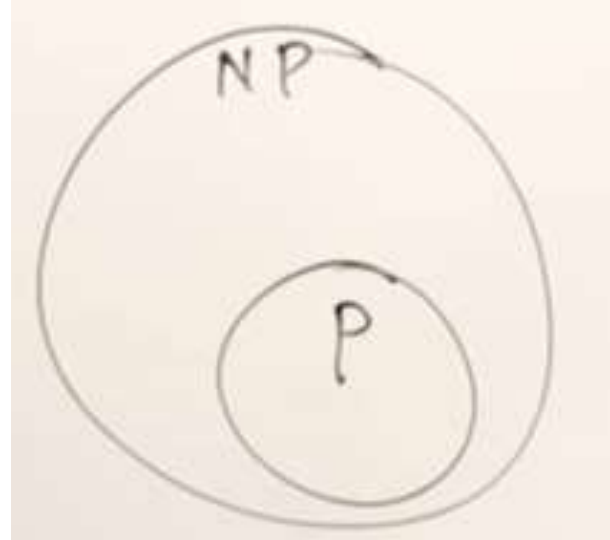
(cont.)

Class *NP*: Non-Deterministic Polynomial-Time Algorithms

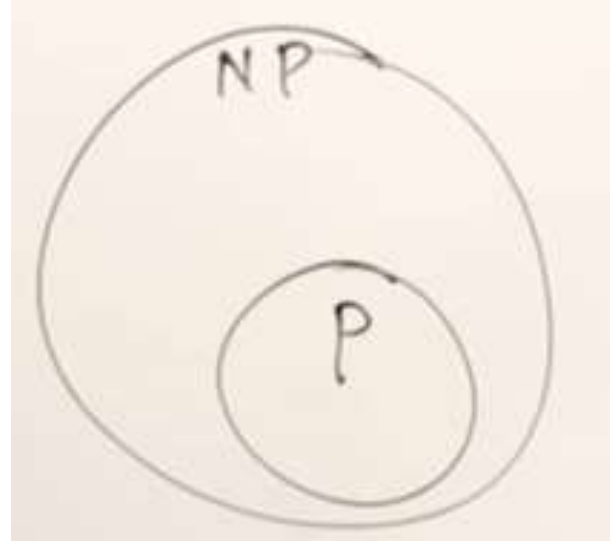
- These are algorithms that:
 - Solve problems in polynomial time, but their implementation may include **non-deterministic steps**.
 - **Non-deterministic steps** are placeholders for logic we currently don't know how to implement but assume can be resolved instantly.
 - These algorithms are typically used for problems currently solved with exponential-time algorithms.
- **Why Use Non-Deterministic Algorithms?**
 - They provide a way to "document" potential solutions for exponential problems, leaving non-deterministic parts for future research.

Relation Between P and NP

- P (deterministic algorithms) is a subset of NP (non-deterministic algorithms).
 - Example: **Merge Sort** ($O(n \log n)$) was once "unknown" and could have been considered non-deterministic until researchers developed it as a deterministic algorithm.
- **Conceptual Idea:**
 - Algorithms in P today might have been in NP in the past.
 - Similarly, non-deterministic algorithms in NP today may become deterministic and move to P in the future as research progresses.



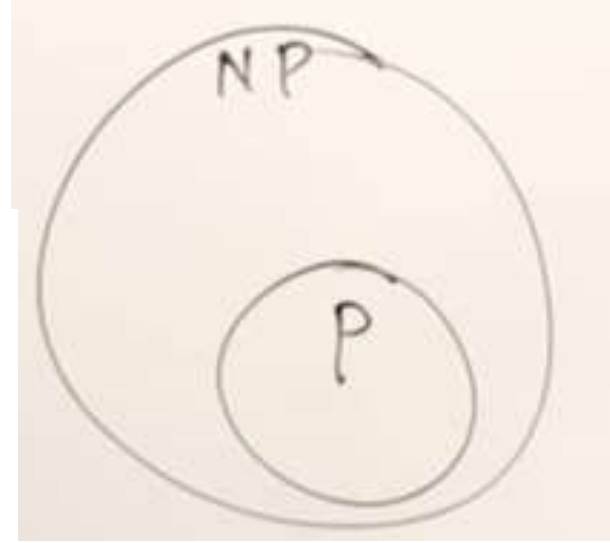
Cook's Theorem:



- Cook's Theorem establishes the foundation of NP -completeness.
- It states that if any NP -complete problem can be solved in polynomial time, then all problems in NP can also be solved in polynomial time, implying $P = NP$.

Summary

1. **Class P :** Deterministic algorithms that take polynomial time.
2. **Class NP :** Non-deterministic algorithms that take polynomial time.
3. **Relationship:** $P \subseteq NP$.
4. **Research Focus:**
 - Write non-deterministic algorithms for exponential problems.
 - Convert these algorithms to deterministic ones to shift problems from NP to P .



The **next** one is:

1. Establishing Relationships Between Problems
for Efficient Solutions

1. Establishing Relationships Between Problems for Efficient Solutions

- When solving **complex** computational problems, we aim to **establish relationships** between them so that solving one can simplify or solve others.
- Instead of addressing each problem individually, we work to **group** and **connect** them **logically**.
- This approach relies on using a **base problem** as a reference, often the **Satisfiability Problem (SAT)**.

(i) . Satisfiability Problem (SAT) : The Base Problem

CNF-Satisfiability
 $x_i = \{x_1, x_2, x_3\}$
 $CNF = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$
 $C_1 \qquad C_2$

$2^3 \Rightarrow 2^n$

x_1	x_2	x_3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

(i) Satisfiability Problem (SAT): The Base Problem

- **Definition:**

SAT is about finding values for Boolean variables ($X_1, X_2, X_3, \dots$) in a given **Conjunctive Normal Form (CNF)** formula such that the formula evaluates to true.

- **Example CNF formula:**

$$\text{CNF} = \text{Product of Sums} \\ (X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_1 \vee X_2 \vee \neg X_3)$$

- Here, the clauses are:

- $C_1 = (X_1 \vee \neg X_2 \vee X_3)$

- $C_2 = (\neg X_1 \vee X_2 \vee \neg X_3)$

- Clauses are combined using AND ($\wedge$), and terms within clauses use OR ($\vee$).

- **Goal:**

Identify combinations of X_1, X_2, X_3 (0 or 1) that satisfy the formula.

- **Complexity:**

- For n variables, there are 2^n possible assignments.
 - Checking all possibilities takes **exponential time** ($O(2^n)$).

CNF-Satisfiability 2^n

$x_i = \{x_1, x_2, x_3\}$

$\text{CNF} = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$

$C_1 \quad C_2$

x_1	x_2	x_3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

$2^3 \Rightarrow 2^n$

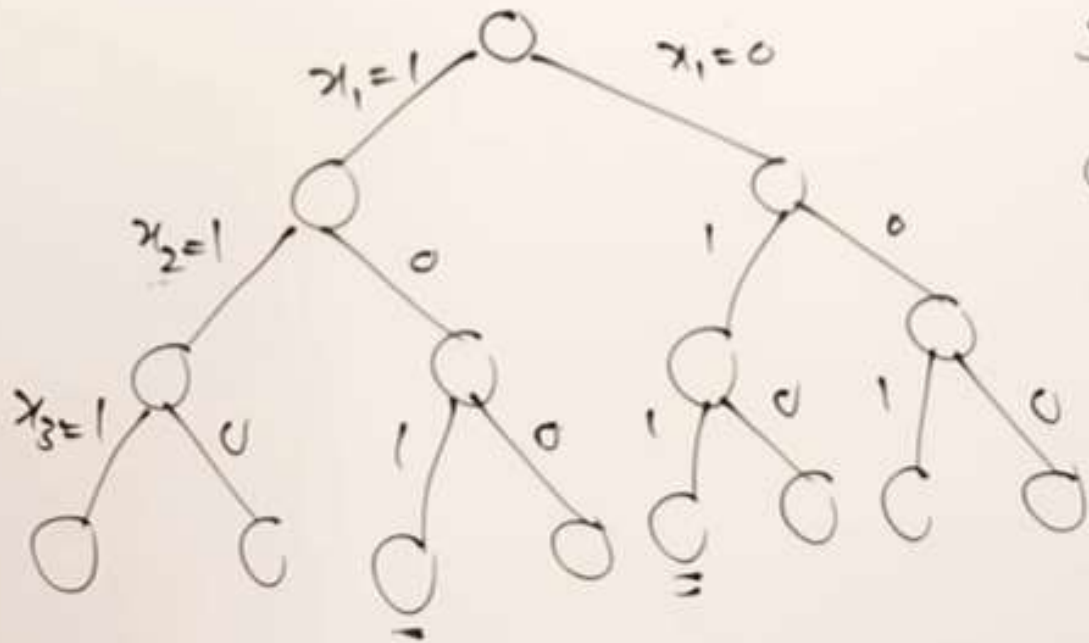
Conjunctive Normal Form (CNF) is a standard way of writing a **Boolean formula** as a **conjunction (AND) of one or more clauses**, where **each clause is a disjunction (OR)**

(11) Representing SAT Using a State Space Tree

CNF-Satisfiability — 2^n

$$x_i = \{x_1, x_2, x_3\}$$

$$\text{CNF} = (\underbrace{x_1 \vee \bar{x}_2 \vee x_3}_{C_1}) \wedge (\underbrace{\bar{x}_1 \vee x_2 \vee \bar{x}_3}_{C_2})$$



Exponential Time

0/1 Knapsack — 2^n

Traveling SP — 2^n

Sum of Subsets — 2^n

Graph Coloring — 2^n

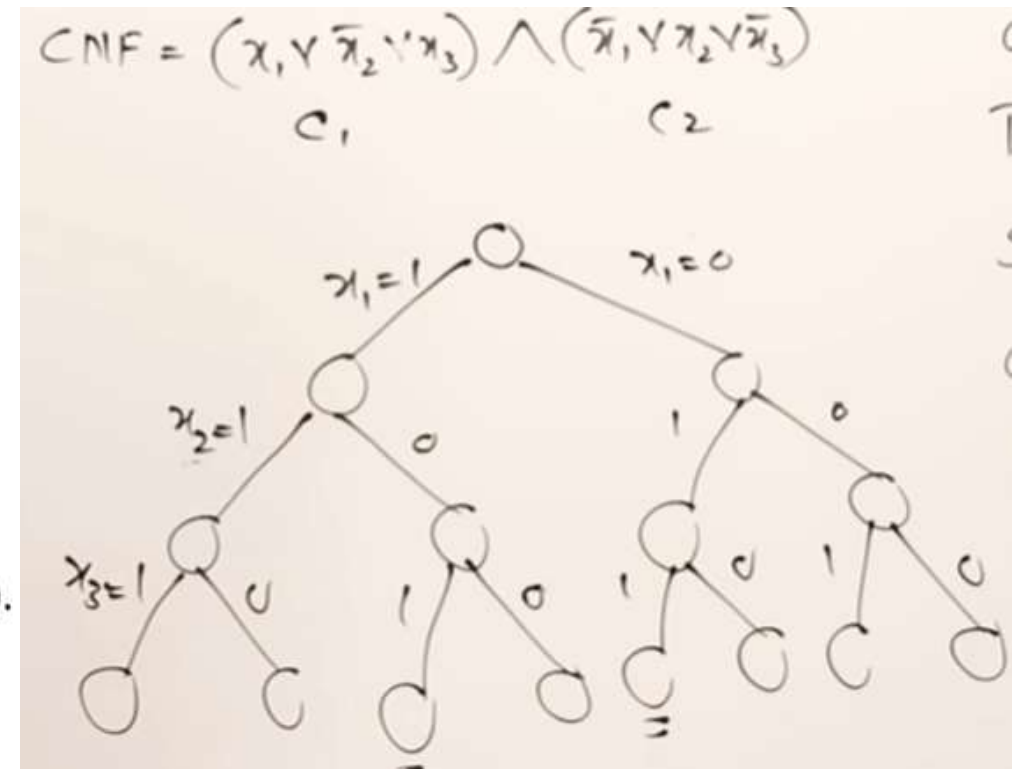
Hamiltonian Cycle — 2^n

(11) Representing SAT Using a State Space Tree

- The solution space for SAT can be visualized as a **state space tree**:
 - Each level of the tree represents a variable ($X_1, X_2, X_3, \dots$).
 - Each node branches into two: 0 (false) and 1 (true).
 - Paths from root to leaves represent all possible combinations of variable assignments.

- Example for three variables (X_1, X_2, X_3):

- Root: Start with no variables assigned.
- First level: $X_1 = 1$ or $X_1 = 0$.
- Second level: $X_2 = 1$ or $X_2 = 0$.
- Third level: $X_3 = 1$ or $X_3 = 0$.
- Leaf nodes represent all 2^n combinations (e.g., 101, 010, ...).



(iii) Relating SAT to Other Problems (e.g., 0/1 Knapsack Problem)

CNF-Satisfiability — 2^n

0/1 Knapsack

$P = \{10, 8, 12\}$ $n=3$
 $W = \{5, 4, 3\}$ $m=8$
 $x_i = \{\underline{0/1}, \underline{0/1}, \underline{0/1}\}$

x_1, x_2, x_3
 $\begin{matrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{matrix}$
 $2^3 \Rightarrow 2^n$

Exponential Time

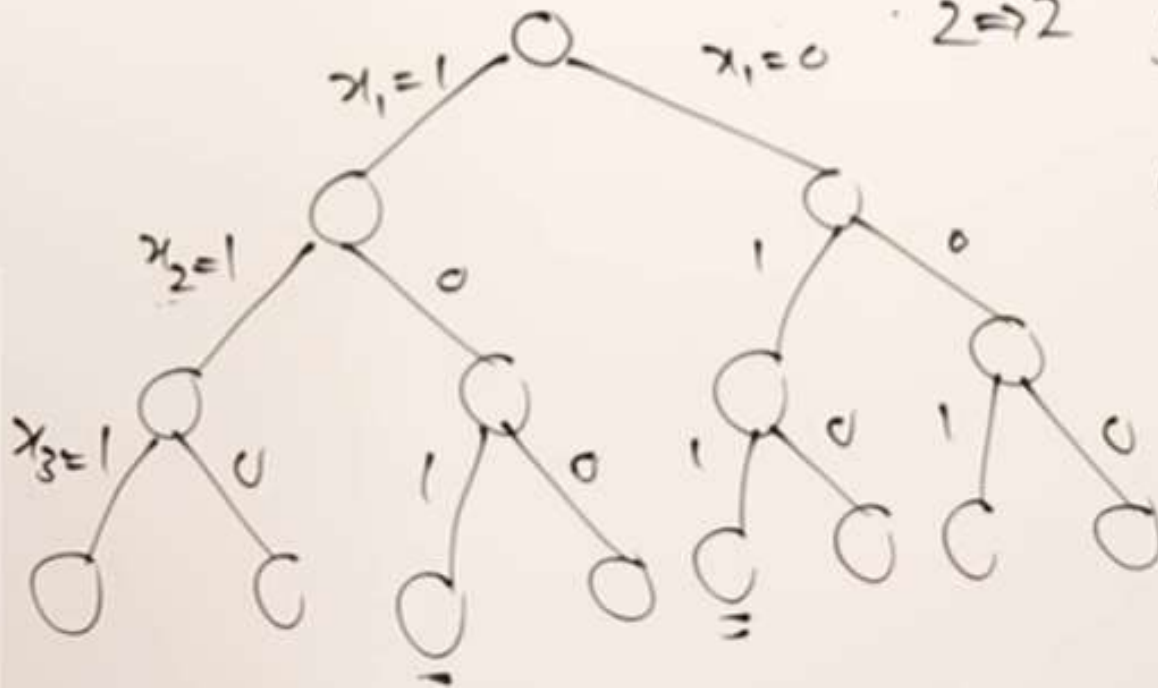
0/1 Knapsack — 2^n

Traveling SP — 2^n

Sum of Subsets — 2^n

Graph Coloring — 2^n

Hamiltonian Cycle — 2^n



(iii) Relating SAT to Other Problems
(e.g., 0/1 Knapsack Problem)

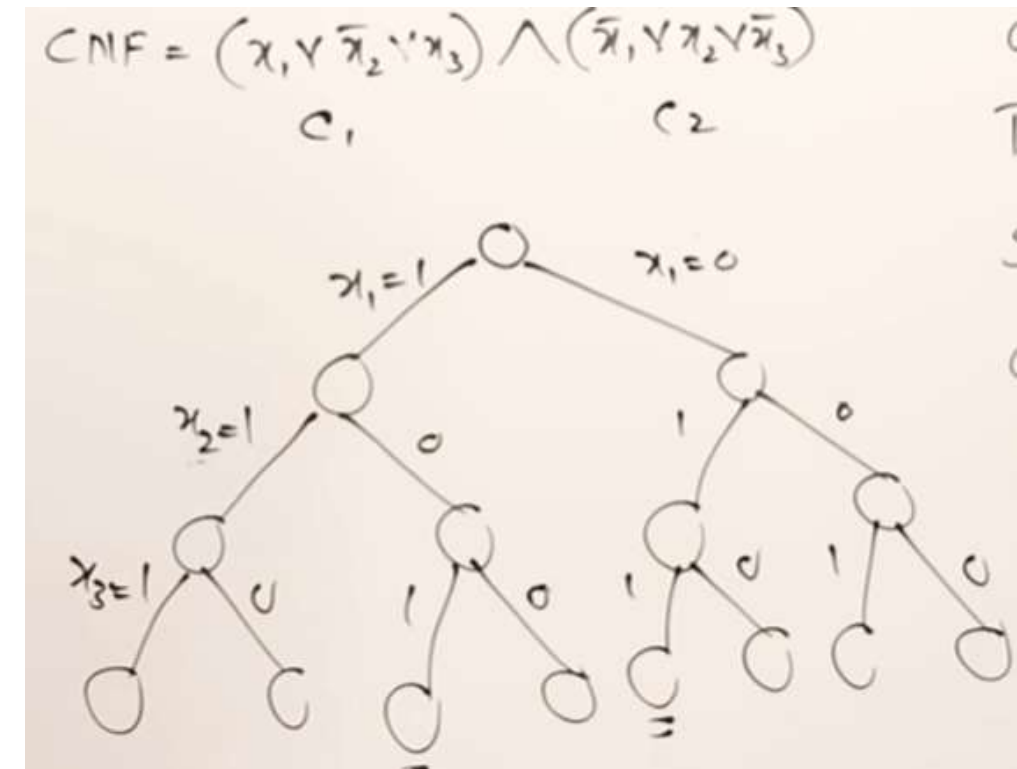
- **0/1 Knapsack Problem Definition:**

- Given n objects, each with a weight and profit, and a bag with a weight capacity:
 - Find a subset of objects to maximize profit without exceeding the bag's capacity.
 - Solutions can be expressed as $X_1, X_2, X_3, \dots$ (0 or 1), where:
 - $X_i = 1$: Include object i in the bag.
 - $X_i = 0$: Exclude object i from the bag.
- $$CNF = (x_1 \vee \bar{x}_2 \vee x_3)$$

C_1

- Similarity to SAT:

- SAT checks if a formula is satisfied(i.e. **True**); 0/1 Knapsack checks if the profit is maximized.
- Both problems involve exploring 2^n possibilities.
- The state space tree for SAT can also represent

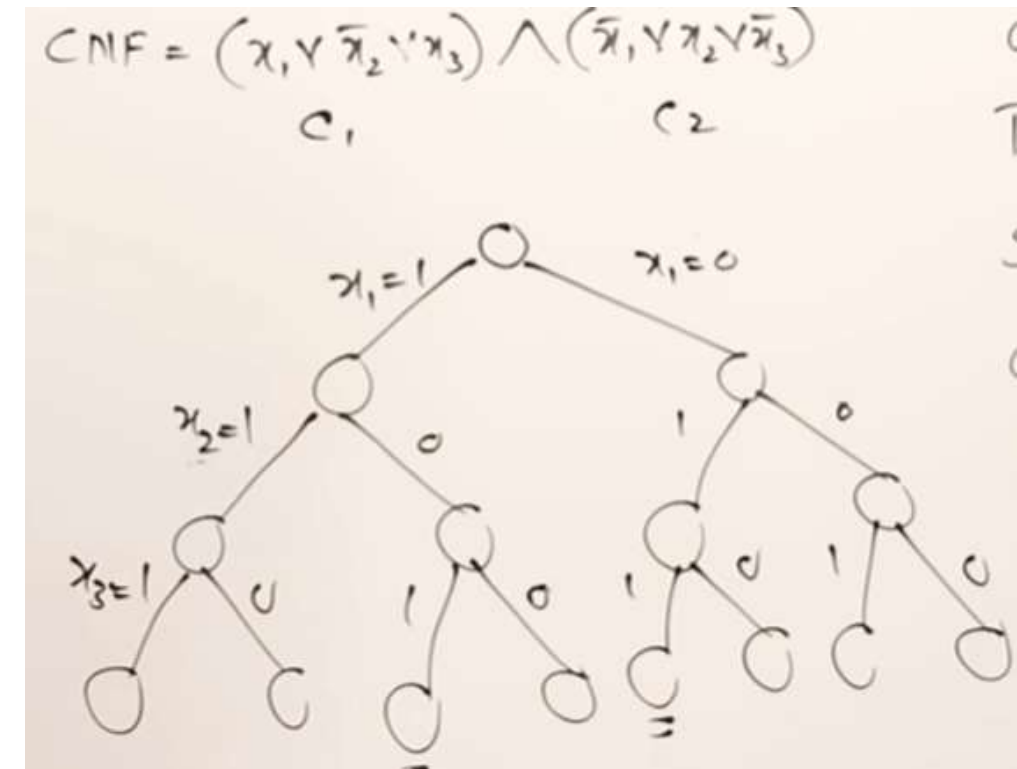


(iii) Relating SAT to Other Problems (e.g., 0/1 Knapsack Problem)

- Key Insight:
 - The same state space tree structure applies to both problems:
 - SAT evaluates clauses for truth.
 - 0/1 Knapsack checks weight and profit constraints.

- Implication:

If SAT can be solved in polynomial time, then 0/1 Knapsack can also be solved in polynomial time, and vice versa.



(iv) Relating Problems Through a Base Problem

- By showing that different problems (e.g., SAT, 0/1 Knapsack) share similar solution structures, we can:
 - Avoid solving each problem independently.
 - Use a solution for one to solve others.
- **How This Helps Research:**
 - If a polynomial-time solution is found for SAT, all problems reducible to SAT (e.g., 0/1 Knapsack) can also be solved in polynomial time.
 - This reduces research effort by focusing on the base problem.

Conclusion: Why Relate Problems?

1. Relating problems simplifies research by avoiding independent solutions.
2. The base problem (SAT) acts as a reference for proving the complexity of other problems.
3. Solving one problem in polynomial time allows solving many others, as they share structural similarities.

Explanation of NP-Hard and NP-Complete Problems

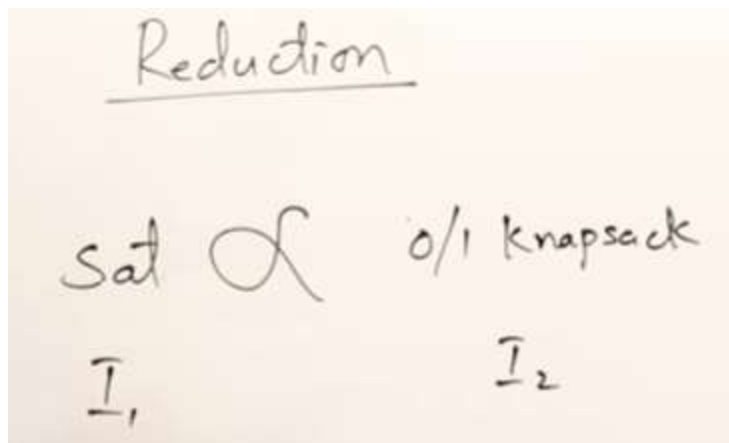
1. Introduction to NP-Hard Problems:

- NP-hard problems are considered *hard problems* because they require **exponential time** to solve using known methods.
- Examples include the **Satisfiability Problem (SAT)**, **0/1 Knapsack Problem**, **Graph Coloring**, and **Hamiltonian Path Problem**.

Explanation of NP-Hard and NP-Complete Problems (cont...)

2. Satisfiability Problem (Base Problem):

- SAT is the **base problem** used to relate other hard problems.
- It involves checking if a given Boolean formula (in Conjunctive Normal Form) can be satisfied by assigning 0 or 1 to its variables.
- For n variables, there are 2^n possible assignments to check.

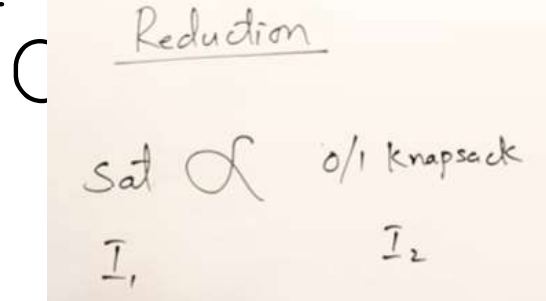


We take some **instance (I_1)** of satisfiability that is

one formula and from that formula, we will convert into a 0/1

knapsack problem

Explanation of NP-Hard and NP-Problems (cont...)



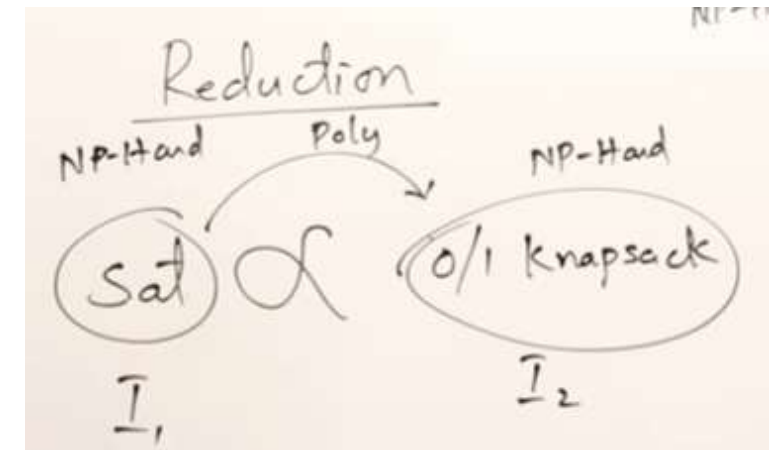
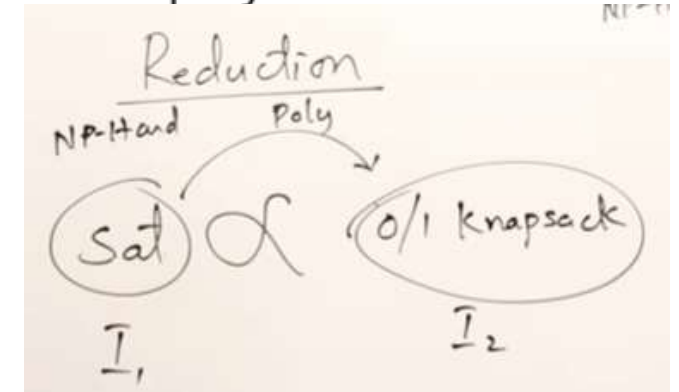
3. Reduction:

- To show a relationship between problems, we use a method called **reduction**.
- Example: To relate SAT to the 0/1 Knapsack Problem:
 - Convert a formula from SAT into a knapsack problem in **polynomial time**.
 - If the knapsack problem can be solved in polynomial time, the same approach can solve SAT in polynomial time and vice versa.
- **Transitive Property of Reduction:** If SAT reduces to Problem A and Problem A reduces to Problem B, then Problem B is also related to SAT.

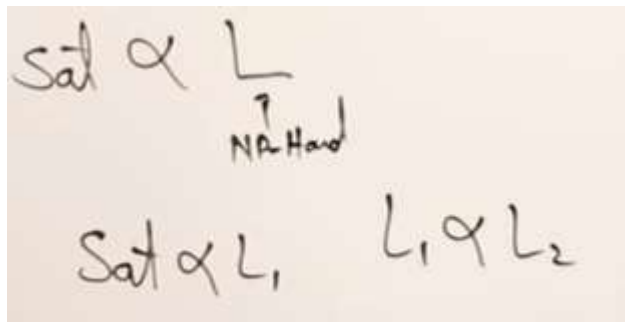
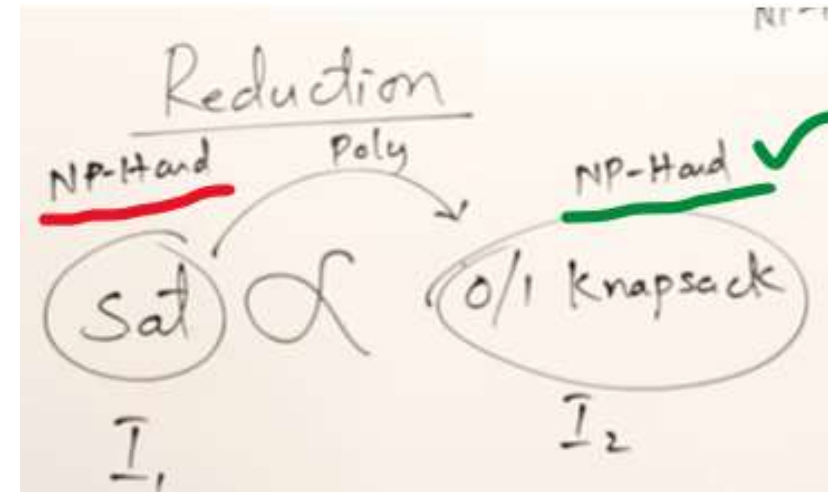
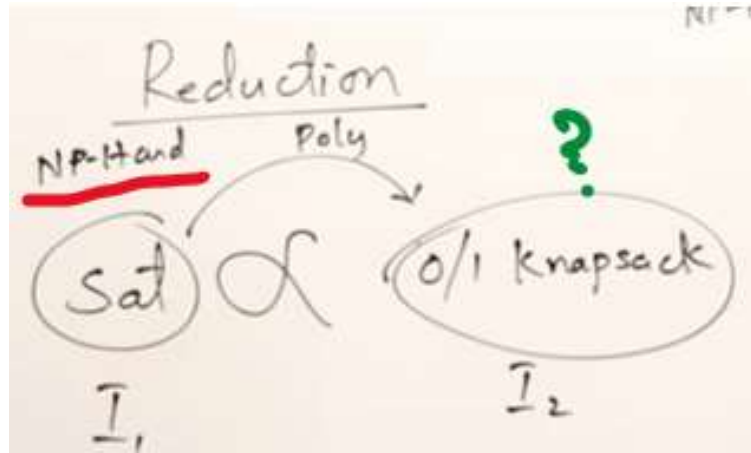
Explanation of NP-Hard and NP-Complete Problems (cont...)

4. NP-Hard and NP-Complete:

- **NP-Hard:** A problem is NP-hard if it is at least as hard as SAT. Solving SAT in polynomial time means all NP-hard problems can be solved in polynomial time.
- **NP-Complete:** A problem is NP-complete if:
 1. It is NP-hard.
 2. It has a **non-deterministic polynomial-time algorithm** (an algorithm that guesses a solution and verifies it in polynomial time).
- SAT is an example of an NP-complete problem because:
 - It is NP-hard.
 - It has a non-deterministic polynomial-time algorithm.



Explanation of NP-Hard and NP-Complete Problems (cont...)



if we have **shown 0/1 knapsack** is **reduced by satisfiability**, then

we can show **graph coloring** reduced by **0/1 knapsack** or

sum of subset reduced by **graph coloring** or
hamiltonian is reduced by **graph coloring**

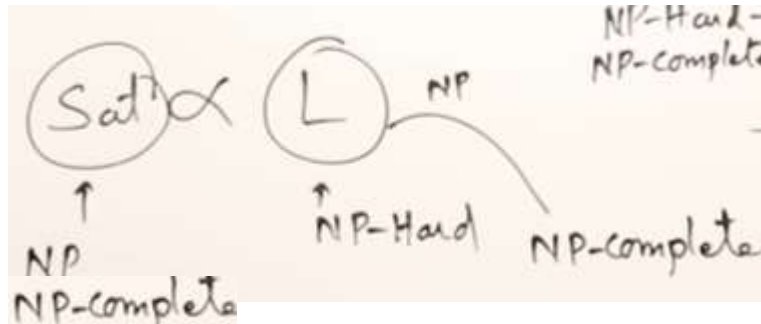
Explanation of NP-Hard and NP-Complete Problems (cont...)

NP-Hard $\rightarrow$ Satisfiability $- 2^n$
NP-complete



satisfiability is NP hard as well as this
is NP complete

If a **problem** is having **non-deterministic
time algorithm** then that problem also
becomes NP complete



So **satisfiability** is a known **NP complete**
for any of these problems, If this
problem (**L**) is reduced by satisfiability,

this **L** is NP hard and if you write a non-
deterministic polynomial time
algorithm, then this **L** also becomes
complete

Exponential Time
0/1 Knapsack $- 2^n$
Traveling SP $- 2^n$
Sum of Subsets $- 2^n$
Graph Coloring $- 2^n$
Hamiltonian Cycle $- 2^n$

Explanation of NP-Hard and NP-Complete Problems (cont...)



On **one part**, this is a set of NP hard problems.

Who is there in this one? **satisfiability**

Then if you prove any of these problem that is reduced by satisfiability

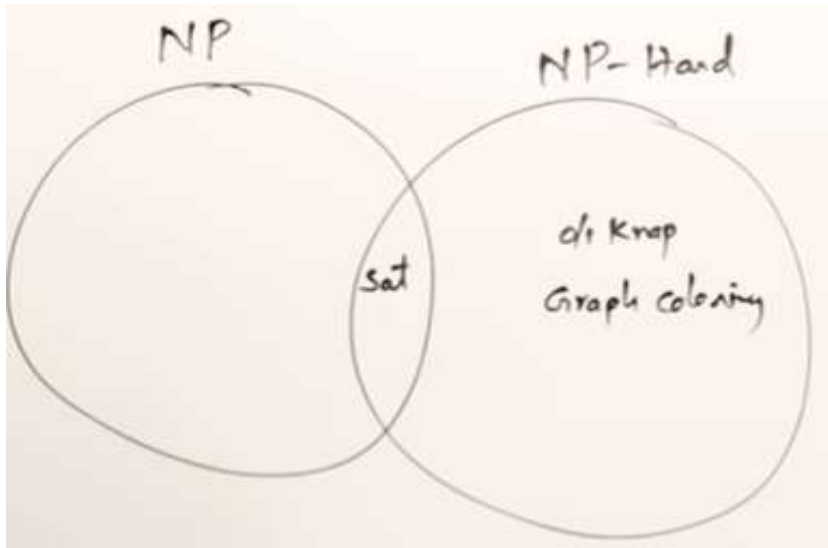
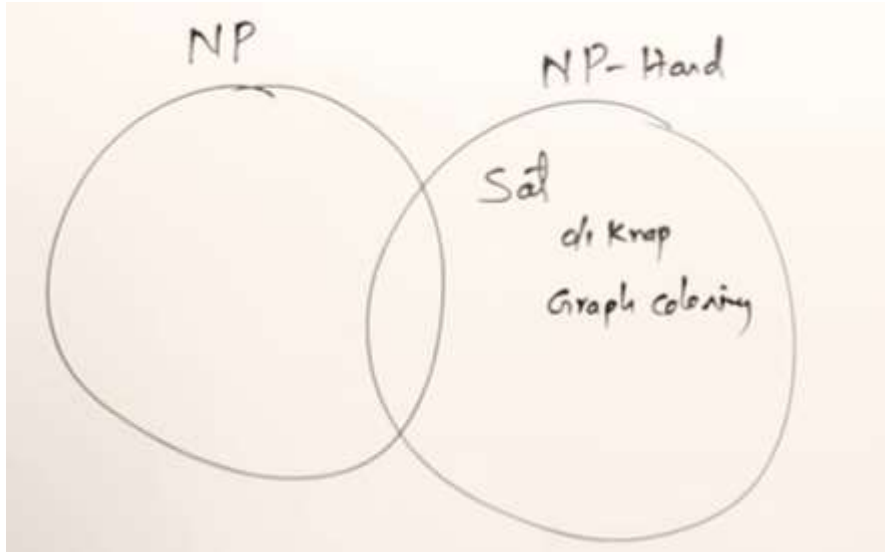
let us say 0/1 knapsack, it also becomes a NP hard

if you prove graph coloring then that is also NP hard

if you are proving that they are directly or indirectly reduced by satisfiability they will become NP hard.



Explanation of NP-Hard and NP-Complete Problems (cont...)



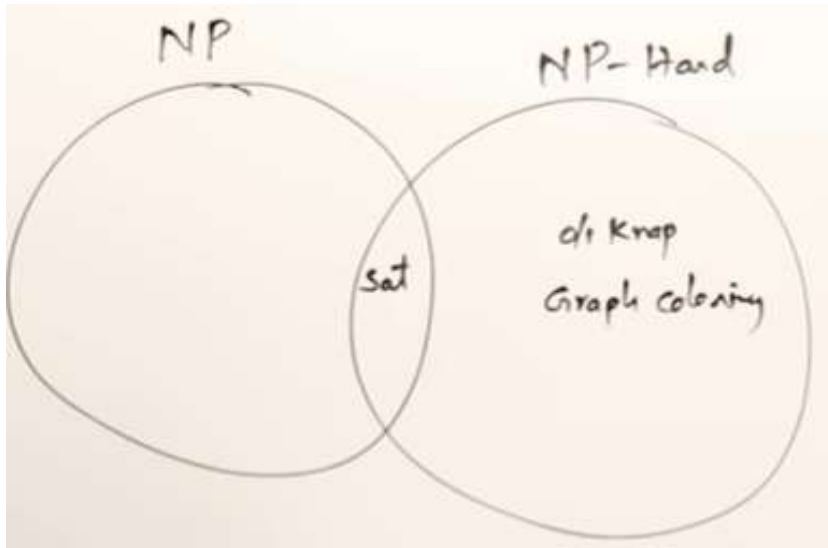
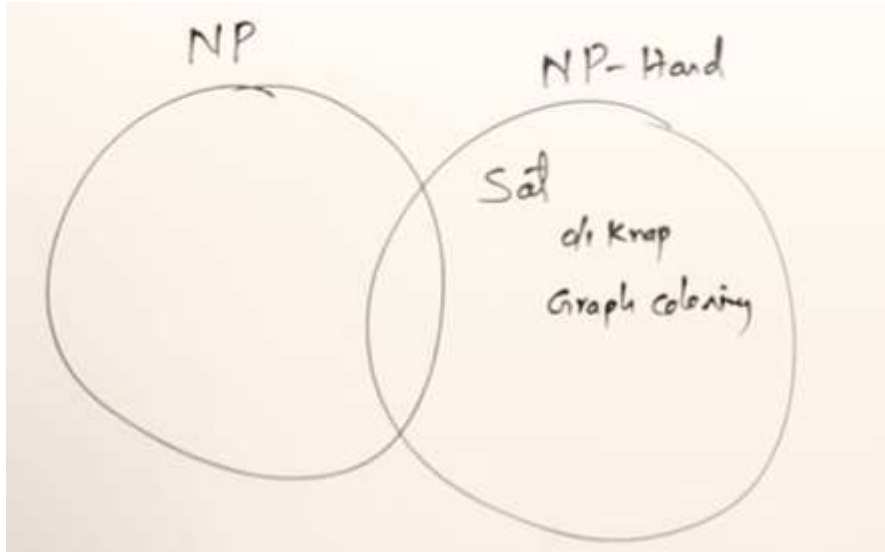
On **second part** , we should also write nondeterministic algorithm NP.

do you have a non-deterministic algorithm for satisfiability? **yes**

so this is in NP hard only.

But this is in this intersection area so this is also NP hard as well as this is having NP (non-deterministic polynomial time) algorithm

Explanation of NP-Hard and NP-Complete Problems (cont...)



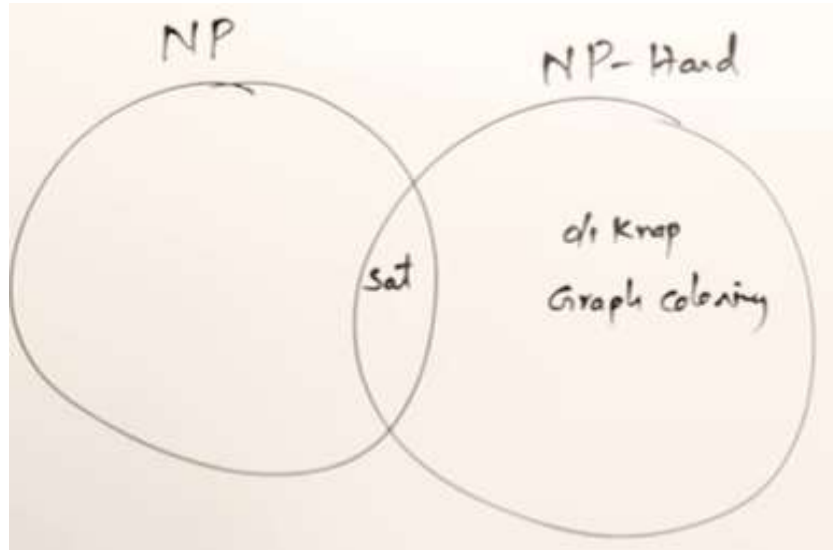
On **second part** , we should also write nondeterministic algorithm NP.

do you have a non-deterministic algorithm for satisfiability? **yes**

so this is in NP hard only.

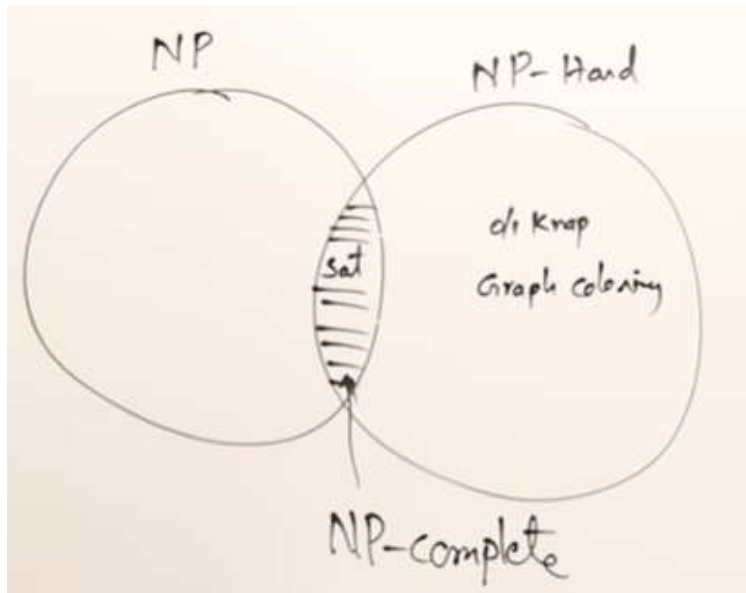
But this is in this intersection area so this is also NP hard as well as this is having NP (non-deterministic polynomial time) algorithm

Explanation of NP-Hard and NP-Complete Problems (cont...)

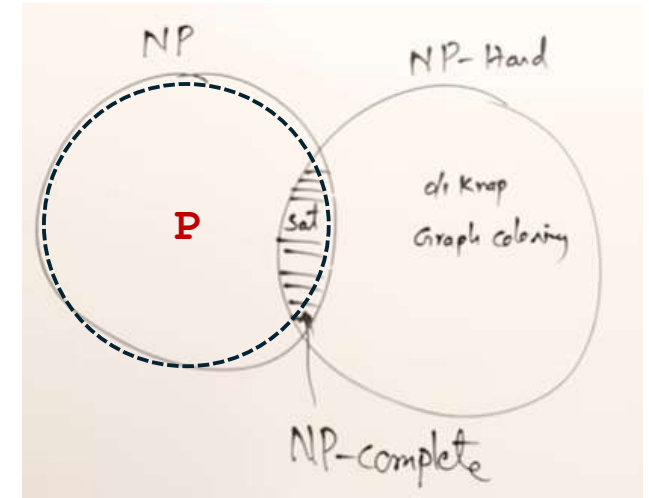
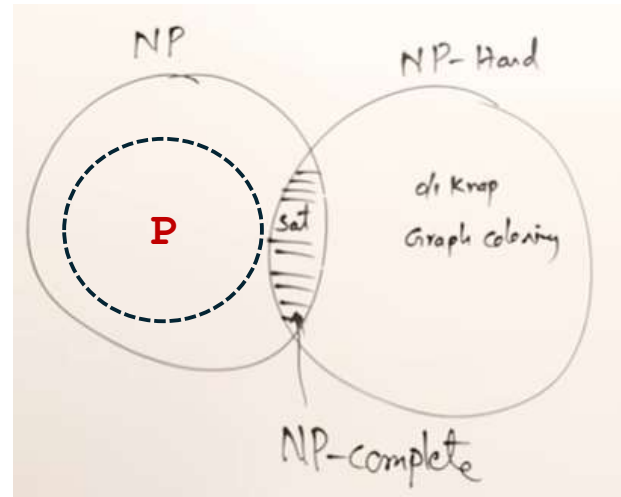
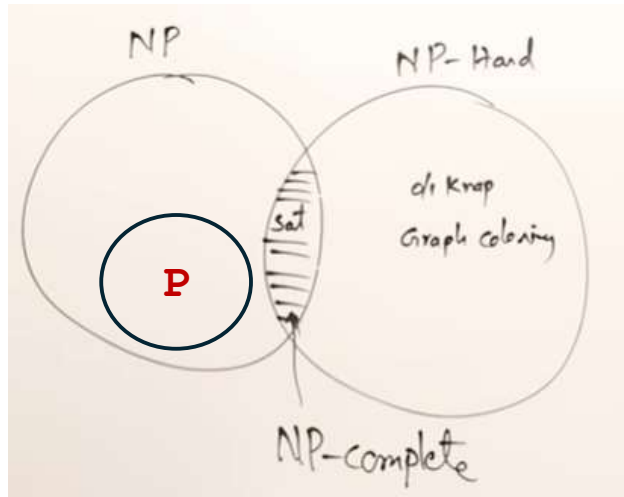


so what is this area called as ? This is NP complete.

→ This is **intersection** of NP hard and NP (non-deterministic polynomial time)



Explanation of NP-Hard and NP-Complete Problems (cont...)

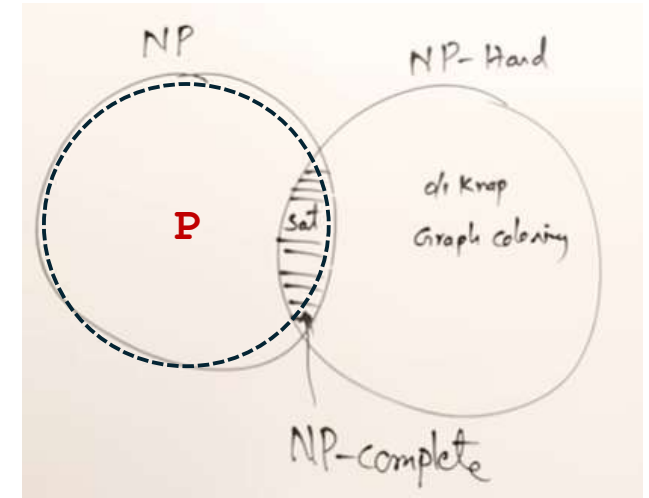
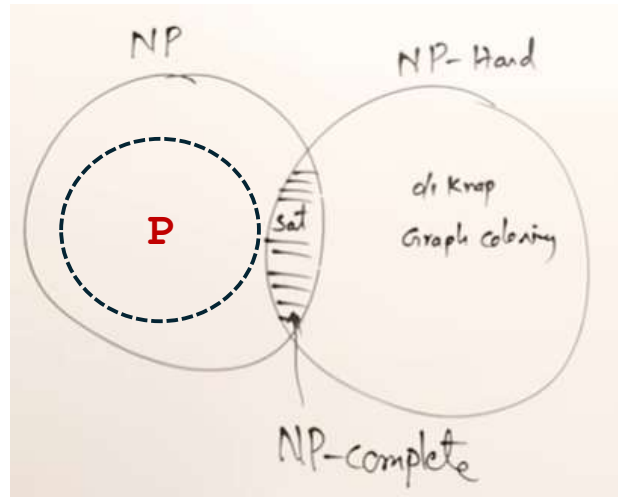
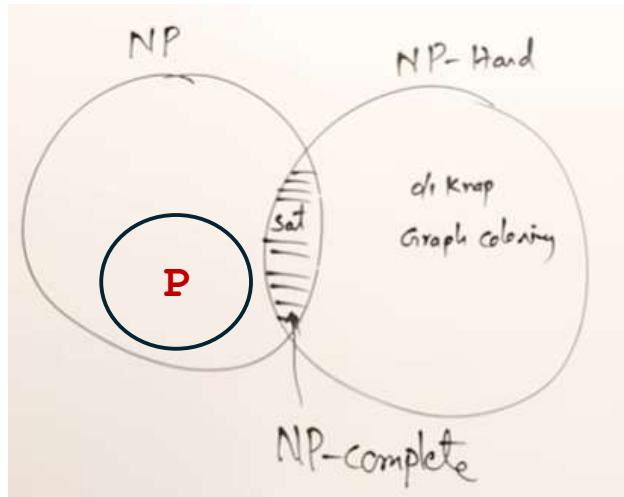


- P is the things that are deterministic today and NPS that is non deterministic.
- Tomorrow, we may be knowing those NPs (non-determinist) also becoming inside p and we believe that this P will be keep on expanding

This P will be keep on growing

P will become bigger and bigger.
At some stage, NP (non-deterministic) is becoming P (deterministic).
Thought: Today we are writing NP which will become P tomorrow. Is it perfect? if it

Explanation of NP-Hard and NP-Complete Problems (cont...)



So, **we need to prove** that really tomorrow we may be knowing non-deterministic algorithm that will be converted into polynomial time deterministic algorithms.

To prove:

If we can prove that $P = NP$, it means that all the non-deterministic algorithms we write today can eventually be turned into deterministic algorithms in the future. Since we already know that P is a subset of NP , we don't know how much overlap exists between them. That's why we aim to show whether $P = NP$ or not. Proving $P = NP$ would confirm that efficient (polynomial-time) solutions already exist for all problems in NP ; we just need to discover them.

Explanation of NP-Hard and NP-Complete Problems (cont...)

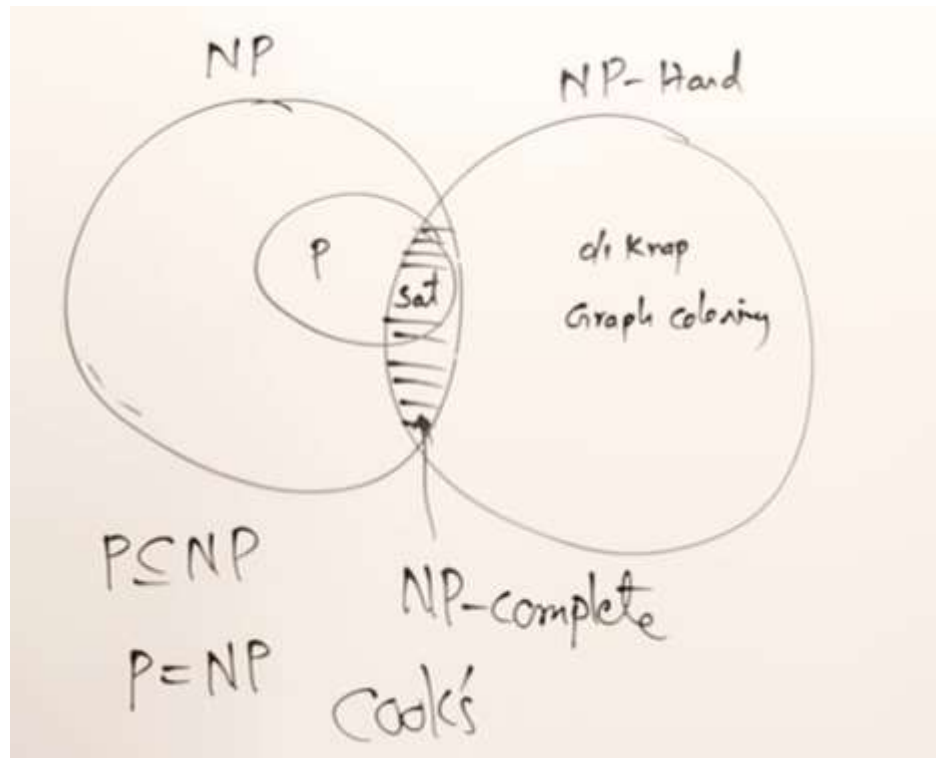
5. Relationship Between P and NP:

- **P:** Problems that can be solved in polynomial time.
- **NP:** Problems whose solutions can be verified in polynomial time.
- We believe $P \subseteq NP$, but we don't know if $P = NP$ (i.e., whether every problem in NP can also be solved in polynomial time).
- If $P = NP$, it means non-deterministic algorithms can eventually be converted into deterministic ones, validating all current research on NP problems.

Explanation of NP-Hard and NP-Complete Problems (cont...)

6. Cook's Theorem:

- Cook proved that if **SAT is in P**, then $P = NP$. In other words, proving SAT is solvable in polynomial time would mean all NP problems are solvable in polynomial time.



- If satisfiability is lying in P so it means **where this p is?**
- We **don't know** even P can be in the **intersection** part of this one.
- So he said that **satisfiability is in P** if and only if p is equal to NP.
- **Satisfiability** will become **deterministic** if p is equal to NP

He stated that this **approach would be validated** if someone could prove that the **satisfiability problem is in P**. While he did not prove it himself, he suggested that if someone does, it would confirm the correctness and completeness of this

Explanation of NP-Hard and NP-Complete Problems (cont...)

7. Diagram Summary:

- **NP-Hard Problems:** All hard problems like SAT, knapsack, graph coloring, etc.
- **NP-Complete Problems:** Intersection of NP-hard problems and problems with non-deterministic polynomial-time algorithms (e.g., SAT).
- P : Problems solvable deterministically in polynomial time.
- We aim to show P and NP might eventually overlap completely ($P = NP$).